8 Pattern təkrarlanan problem üçün həll quruluşudur

Kafe tətbiqində qəbzin mətn və PDF variantı, çatdırılmanın adi və sürətli variantı, ödənişin müxtəlif üsulları yarandı. Hər yerdə eyni dəyişiklik problemlərini təkrar görürük.

Design pattern belə təkrarlanan problemə verilən ümumi həll quruluşudur. Hazır köçürüləcək kod deyil. Onun adını bilməzdən əvvəl həll etdiyi problemi başa düşmək lazımdır.

Dəyişən hissəni əvvəl seç

```
Hansi davranis deyisir?       -> Strategy dusun.
Hansi obyekt alinmalidir?     -> Factory dusun.
Movcud interfeys uygun deyil? -> Adapter dusun.
Eyni ise elave davranis?      -> Decorator dusun.
```

Bu adlar seçimə kömək edir. Eyni quruluşu hər problemə tətbiq etmək lazım deyil.

Klassik **GoF** kataloqunda 23 pattern var. GoF həmin kitabın dörd müəllifinə verilən addır. Pattern-lər üç ailəyə ayrılır: **creational** yaradılmanı, **structural** quruluşu, **behavioral** davranış və ünsiyyəti təşkil edir.

Bu dərsdə əvvəl backend-də asan rast gəlinən problemləri açacağıq. Sonra qalan pattern-ləri konkret nümunə ilə bir-birinə bağlayacağıq.

Strategy dəyişən hesabı ayrıca saxlayır

Adi müştəri və VIP üçün fərqli endirim hesablayırıq. Məqsəd hər ikisində eynidir: endirim məbləğini tapmaq. Dəyişən hissə hesablama qaydasıdır.

SOLID dərsindəki `IDiscount` nümunəsini xatırla. Hər variant `Calculate(total)` təqdim edir. Əsas sifariş kodu seçilmiş variantı çağırır:

C# istifadə fraqmenti

```
decimal discountAmount = discount.Calculate(total);
decimal payable = total - discountAmount;
```

`discount` constructor-la verilmiş IDiscount obyektidir. `discountAmount` endirim məbləği, `payable` ödəniləcək məbləğdir.

Bu quruluş **Strategy** pattern-dir. Əsas kod eyni məqsədli alqoritmlərdən birini istifadə edir. **Alqoritm** nəticəyə çatmaq üçün addımlar qaydasıdır.

Əlaqə

```
Checkout -> IDiscount
                |
                +-- NormalDiscount
                +-- VipDiscount
                +-- StudentDiscount
```

Yeni endirim gələndə onun hesabı ayrıca yazılır. Checkout-un əsas hesablaması dəyişməyə bilər. Hansı strategiyanın seçilməsi isə ayrıca seçim yerinin işidir.

Nə vaxt uyğundur: eyni işi görən qaydaların variantları artır və müstəqil dəyişir. Cəmi iki sabit, sadə hal varsa bir şərt ifadəsi də kifayət edə bilər. Pattern-in əlavə quruluş xərci də var.

Factory hansı obyektin alınacağını bir yerə toplayır

Müştəri qəbzi PDF istəyir, kassir isə mətn qəbzi istəyir. Hər istifadə yerində “PDF-dirsə bunu yarat, mətndirsə onu yarat” kodunu təkrarlamaq istəmirik.

Factory obyekt yaratma və ya uyğun obyekti vermə məntiqini ayrıca yerə toplayır. Sadə seçim metoduna çox vaxt **Simple Factory** deyilir.

Sadə factory fikri

```
CreateReceipt("text") -> TextReceipt obyekti.
CreateReceipt("pdf")  -> PdfReceipt obyekti.
```

Klassik **Factory Method** bir qədər fərqlidir: baza sinif ümumi işi saxlayır, hansı məhsul obyektinin yaradılacağını alt sinfin əvəz edə bildiyi metoda verir.

Factory Method əlaqəsi

```
ReportJob: "Exporter yarat, sonra export et."
    |
    +-- CsvJob CreateExporter metodunu deyisir.
    +-- PdfJob CreateExporter metodunu deyisir.
```

Baza sinif ortaq davranışı təqdim edən sinifdir. **Alt sinif** ondan törəyib müəyyən addımı dəyişən sinifdir. Export məlumatı başqa formata çıxarmaqdır. CSV sadə cədvəl mətn formatıdır.

Hər adı Create olan metod GoF Factory Method deyil. Əsas məntiq yaradılma qərarının alt sinfə verilən dəyişmə nöqtəsi olmasıdır.

Strategy ilə fərq: Strategy davranışın necə hesablanmasına, Factory isə hansı obyektin yaradılıb verilməsinə aiddir. Birlikdə işləyə bilərlər.

Abstract Factory uyğun məhsul ailəsi yaradır

Kafe proqramının açıq və tünd görünüşü var. Hər görünüşün düyməsi və seçim qutusu bir-birinə uyğun olmalıdır.

Bir factory yalnız bir obyekt yox, uyğun bir neçə obyekt yarada bilər:

Məhsul ailəsi

```
LightFactory
    CreateButton()   -> aciq duymə.
    CreateCheckbox() -> aciq secim qutusu.

DarkFactory
    CreateButton()   -> tund duymə.
    CreateCheckbox() -> tund secim qutusu.
```

Bu **Abstract Factory** pattern-dir. Məqsəd uyğun məhsullar ailəsini birlikdə yaratmaqdır. Onu sadəcə “factory-lərin idarəçisi” kimi əzbərləmək əsas məntiqi gizlədir.

Yeni görünüş ailəsi əlavə etmək rahatlaşır. Amma ailəyə yeni məhsul növü, məsələn slider, əlavə etsən bütün factory-lərin müqaviləsi dəyişə bilər. Slider dəyəri sürüşdürərək seçən görünüş elementidir.

DI ilə əlaqə: container obyektlərin əlaqələrini və lifetime-ını qura bilər. Factory isə konkret girişə görə uyğun variantın alınmasını aydın ifadə edə bilər. .NET DI da seçim funksiyası və açarlı xidmətlər kimi üsullar təqdim edir. “DI işləmə vaxtında seçim edə bilmir” düzgün iddia deyil.

Əsas sual ad deyil: bir obyekt seçirsən, alt sinfə yaradılma addımı verirsən, yoxsa uyğun məhsul ailəsi yaradırsan? Bu fərq factory növünü başa salır.

Adapter uyğun gəlməyən çağırışı çevirir

Köhnə printer kitabxanasında metod `SendText` adlanır. Yeni sifariş kodumuz isə `IPrinter.Print` müqaviləsi gözləyir. İş eynidir, giriş formaları uyğun deyil.

Adapter bu uyğunlaşdırmanı ayrıca obyektə yığır:

C#

```
public class PrinterAdapter : IPrinter
{
    private LegacyPrinter oldPrinter;

    public PrinterAdapter(LegacyPrinter oldPrinter)
    {
        this.oldPrinter = oldPrinter;
    }

    public void Print(string text)
    {
        oldPrinter.SendText(text);
    }
}
```

Bu fraqmentdə IPrinter-in `void Print(string text)` müqaviləsini, LegacyPrinter-in isə `SendText(string text)` metodunu təqdim etdiyini qəbul edirik. Legacy köhnədən qalmış sistem deməkdir.

Sifariş kodu IPrinter-i tanıyır. Adapter çağırışı köhnə metodun formasına çevirir. Köhnə kitabxananı bütün tətbiqə yaymırıq.

Uyğunlaşdırma yalnız metodun adını dəyişmək olmaya bilər. Məsələn, yeni müqavilə manat, köhnə sistem qəpik qəbul edirsə məbləğ və yuvarlaqlaşdırma da düzgün çevrilməlidir.

Yadda saxla: Adapter-in əsas niyyəti fərqli müqavilələri uyğunlaşdırmaqdır.

Decorator eyni müqaviləyə əlavə davranış verir

Qiymət xidmətimiz işləyir. İndi hər qiymət istəniləndə bir qeyd yazmaq istəyirik. Əsas qiymət hesabını dəyişmədən onu başqa obyektlə sarıya bilərik.

C#

```
public class LoggedPrice : IPrice
{
    private IPrice inner;

    public LoggedPrice(IPrice inner)
    {
        this.inner = inner;
    }

    public decimal Get(int id)
    {
        Console.WriteLine("Qiymet istendi");
        return inner.Get(id);
    }
}
```

IPrice müqaviləsində `decimal Get(int id)` metodu olduğunu qəbul edirik. `inner` əsl işi görən qiymət xidmətidir. Yeni obyekt eyni müqaviləni saxlayır, çağırışa qeyd əlavə edir və işi inner-ə ötürür. Bu ötürməyə **delegation** deyilir.

Bu **Decorator** pattern-dir. Eyni yanaşma ölçmə və uyğun hallarda keş üçün işlədilə bilər. **Keş** əvvəl alınmış nəticəni təkrar istifadə üçün saxlamaqdır; köhnəlmiş nəticənin nə vaxt silinəcəyi ayrıca seçilir.

Adapter ilə fərq: Adapter müqaviləni uyğunlaşdırır. Decorator eyni müqavilə ətrafında əlavə davranış qurur.

Mediator əlaqələndirməni vasitəçiyə verir

Sifariş yaradılır. Ehtiyat yoxlanmalı, ödəniş təşkil olunmalı, qəbz hazırlanmalıdır. Hər obyekt bütün digərlərini birbaşa tanıyırsa əlaqə şəbəkəsi çətinləşə bilər.

Mediator qarşılıqlı işi əlaqələndirən vasitəçidir. Komponentlər bir-birinin bütün təfərrüatını bilmədən onun üzərindən işləyə bilər.

Sadələşdirilmiş əlaqə

```
Sifaris isteyi
    |
    v
Vasiteci
    +-- uygun isleyiciye yoneldir.
    +-- lazim olan elaqelendirməni aparir.
```

Konkret işi emal edən hissəyə **handler**, işləyici deyilir. .NET-də MediatR adlı kitabxana var, amma pattern-in özü həmin kitabxanadan asılı deyil.

Mühüm sərhəd var: A vasitəçi ilə B-ni, B isə vasitəçi ilə yenə A-nı çağırırsa çağırış dövrəsi yenə qala bilər. Sadəcə araya vasitəçi qoymaq səhv məsuliyyət bölgüsünü avtomatik düzəltmir.

Vasitəçi bütün biznes işlərini özünə yığıb həddindən artıq böyük obyektə də çevrilməməlidir.

Nəticə: Mediator əlaqələri idarə etməyi mərkəzləşdirə bilər. Hər dairəvi asılılığın və hər mürəkkəb sistemin avtomatik həlli deyil.

State Strategy və Template Method fərqi

State cari vəziyyətə görə davranışı dəyişir

Sifariş əvvəl Pending, sonra Paid, sonra Shipped olur. Bunlar gözləmədə, ödənilib və göndərilib vəziyyətləridir. Göndərilmiş sifarişdə bəzi əməliyyatlara artıq icazə verilmir. **State** hər vəziyyətin davranışını ayrıca təşkil edir. Bir vəziyyətdən başqasına keçidə **transition** deyilir.

Strategy-də əsas sual “hansı hesablama üsulunu seçək?” idi. State-də “obyekt hazırda hansı vəziyyətdədir və nə edə bilər?” sualıdır.

Template Method ardıcıllığı sabit saxlayır

Fayl idxalı həmişə oxu, yoxla, çevir, saxla addımlarından keçir. CSV və JSON üçün oxuma addımı fərqlənə bilər. **Template Method** ümumi ardıcıllığı baza sinifdə saxlayır, müəyyən addımları alt sinfə verir. JSON məlumatın açar və dəyərlərlə yazıldığı mətn formatıdır.

Command əməliyyat istəyini obyekt edir

“Sifarişi ləğv et” istəyini əməliyyat növü və sifariş nömrəsi olan obyekt kimi saxlaya bilərik. **Command** həmin istəyi sonradan işlətməyə, növbəyə qoymağa və qeyd etməyə uyğun edir. **Növbə** işlərin sonradan işlənmək üçün saxlandığı ardıcıllıqdır.

Command yaratmaq hər işi geri almağı avtomatik mümkün etmir. Göndərilmiş email üçün sadə undo yoxdur.

Digər struktur patternlərini nümunə ilə ayıraq

Bridge iki ayrı dəyişən istiqaməti ayırır

Bildirişin növü sadə və təcili ola bilər. Göndərmə kanalı email və SMS ola bilər. Hər kombinasiyaya ayrıca sinif yaratmaq əvəzinə bildiriş məntiqi ilə kanalı ayrı saxlayırıq. Bu **Bridge** ideyasıdır. İki tərəf ayrıca genişlənə bilər.

Composite tək elementlə ağacı eyni qayda ilə işlədir

Bir menyu bölməsində məhsullar və başqa alt bölmələr ola bilər. Həm tək məhsula, həm bütöv bölməyə eyni “göstər” əməliyyatı tətbiq etmək istəyirik. **Composite** element və element qrupuna ortaq müqavilə verir. İç-içə qruplar ağac quruluşudur.

Facade mürəkkəb alt sistemi sadə girişlə təqdim edir

Sifariş tamamlamaq üçün bir neçə texniki hissə çağırılmalıdır. **Facade** istifadəçiyə sadə giriş verib həmin təfərrüatları arxada əlaqələndirir. “CompleteOrder” çağırışı bunun nümunəsi ola bilər; transaction zəmanəti ayrıca qurulur.

Flyweight dəyişməz ortaq məlumatı paylaşır

Minlərlə simvol eyni şrift məlumatını ayrıca saxlamaq əvəzinə paylaşa bilər. **Flyweight** belə ortaq hissəni ayırır. Hər simvolun ekrandakı yeri isə ayrıca verilir. Paylaşılan məlumatı nəzarətsiz dəyişmək düzgün deyil.

Proxy obyektə çıxışı idarə edir

Proxy əsl obyektin qarşısında dayanıb giriş icazəsini, uzaq çağırışı və ya gec yaradılmanı idarə edə bilər. Quruluşu Decorator-a bənzəyə bilər. Burada əsas niyyət obyektə çatmağı idarə etməkdir.

Yaradılma və vəziyyət saxlama üçün digər vasitələr

Builder mürəkkəb nəticəni addım-addım qurur

Hesabatın başlığı, bölmələri və filtrləri verilir, sonda Build çağırılır. **Builder** qurulmanı mərhələlərə ayırır və tamamlananda məcburi hissələri yoxlaya bilər. Metodları zəncirlə yazmaq təkbaşına Builder olması demək deyil; əsas niyyət etibarlı nəticə qurmaqdır.

Prototype hazır nümunədən kopya yaradır

Mürəkkəb şablonu sıfırdan qurmaq əvəzinə kopyalaya bilərik. **Prototype** buna xidmət edir. Daxili siyahıları da eyni istinadla paylaşan kopya **shallow copy**-dir. Onları ayrıca kopyalamaq **deep copy** yanaşmasıdır. Hansı hissənin paylaşılacağı düşünülməlidir.

Singleton seçilmiş mühitdə bir nüsxəni paylaşır

Klassik **Singleton** bir nüsxə və ona çıxışı təşkil edir; çox vaxt proqramın hər yerindən əlçatan qlobal girişlə yazılır. DI singleton isə bir container-də paylaşılma qaydasıdır. Bunları eyniləşdirmə. İki prosesin ayrıca nüsxəsi olur. Nüsxənin təhlükəsiz yaradılması onun bütün metodlarının paralel istifadəyə təhlükəsiz olduğunu sübut etmir.

Iterator daxili saxlanmanı açmadan gəzməyə imkan verir

`foreach` siyahının elementlərini gəzir. **Iterator** istifadəçiyə elementləri ardıcıl əldə etməyə imkan verir; onların daxildə massiv, ağac və ya başqa quruluşda olması gizli qala bilər.

Memento əvvəlki vəziyyəti saxlayır

Redaktorda son dəyişikliyi geri almaq üçün əvvəlki vəziyyətin surətini saxlayırıq. **Memento** bunu obyektin daxili quruluşunu hər yerə açmadan təşkil edir. Böyük surətlərin yaddaş xərci vardır.

Hadisə zəncir və qayda emalı

Observer dəyişiklik barədə abunə olanlara xəbər verir

Qiymət dəyişəndə ekran və başqa izləyicilər yenilənsin. **Observer** hadisəni yayan obyektlə həmin hadisəyə abunə olanları əlaqələndirir. C# event mexanizmi bu ideyaya nümunədir. Uzunömürlü yayıcı artıq lazım olmayan abunəçini saxlaya bilər; abunəliyin bitirilməsi nəzərə alınır.

Chain of Responsibility sorğunu işləyicilərdən keçirir

Sorğu A, B və C mərhələlərindən keçir. B işi tamamlayıb dayandıra və ya C-yə ötürə bilər. **Chain of Responsibility** bu quruluşdur. Middleware dərsində gördüyümüz axın buna yaxın nümunədir. Sıra dəyişəndə nəticə də dəyişə bilər.

Interpreter kiçik bir dilin qaydalarını işlədir

`total > 100 AND vip` kimi sadə qaydanın mənasını müəyyən edib qiymətləndiririk. **Interpreter** həmin dilin hissələrini təmsil edir və şərh edir. **Grammar** hansı ifadələrin düzgün qurulduğunu bildirən qaydalardır. Çox böyük dil üçün bu quruluş ağırlaşa bilər.

Visitor sabit obyekt quruluşuna yeni əməliyyat verir

Məlumat ağacındakı tiplər sabitdir, amma onları çap etmək, hesablamaq və yoxlamaq kimi əməliyyatlar artır. **Visitor** əməliyyatı həmin tiplərdən ayrı təşkil edir. Yeni əməliyyat əlavə etmək rahatlaşa, yeni obyekt tipi əlavə etmək isə bütün visitor-lərə təsir edə bilər.

Nəticə: bu pattern-lərin hamısı “kod səliqəli olsun” deyə eyni işi görmür. Hərəsi konkret dəyişmə problemini başqa yerdə saxlayır.

Backend-də eşidəcəyin əlavə patternlər

Bunlar klassik 23 GoF pattern ilə eyni siyahı deyil, amma müsahibədə tez-tez yanaşı çəkilir.

Repository domain obyektlərinə çıxışı bir sərhəddə toplayır. Domain biznes anlayışları və qaydalarıdır. “Müştərinin aktiv sifarişlərini ver” əməliyyatı saxlanma təfərrüatından ayrılmış müqavilə ola bilər.

Unit of Work bir iş ərzində dəyişən obyektləri izləyib yazmaları əlaqələndirir. EF Core DbContext bu imkanların çoxunu artıq verir. Üzərinə əlavə ümumi repository yazmaq hər layihədə məcburi deyil.

CQRS dəyişiklik əmrləri ilə oxuma sorğularının məsuliyyətini və modelini ayırır. Ayrı baza və ayrı mikroservis məcburi deyil. Əsas fikir yazma modeli ilə oxuma ehtiyacının həmişə eyni formaya sıxışdırılmamasıdır.

Outbox sifariş dəyişikliyi ilə göndəriləcək hadisəni eyni baza transaction-unda saxlayır. Ayrı işçi sonradan hadisəni göndərir. Bu, “sifariş yazıldı, xəbər itdi” problemini həll etməyə kömək edir. Amma hadisə təkrar göndərilə bilər.

Idempotency eyni iş təkrar gələndə əlavə yanlış təsir yaratmamaqdır. Məsələn, eyni hadisə nömrəsi ilə iki dəfə ödəniş yazmamaq. “SaveChanges et, sonra Publish et” təkbaşına iki ayrı sistem arasında birlikdəlik zəmanəti yaratmır. Publish hadisəni yaymaq deməkdir.

Adı deyil problemi tanımaq üçün məşq

Problem	Düşünüləcək pattern
Yeni endirim hesabı əlavə olunur	Strategy
Uyğun qəbz obyektini bir yerdən almaq lazımdır	Factory yanaşması
Köhnə printerin metodu yeni müqaviləyə uyğun deyil	Adapter
Əsas qiymət koduna toxunmadan qeyd əlavə edilir	Decorator
Obyektin vəziyyəti hansı əməliyyatın mümkün olduğunu dəyişir	State
Mürəkkəb alt sistemə sadə giriş lazımdır	Facade

Özünü yoxla

“Adapter ilə Decorator eyni deyil” cümləsinin səbəbini de. **Adapter fərqli müqaviləni uyğunlaşdırır; Decorator eyni müqavilə ətrafına davranış əlavə edir.**

“Strategy ilə Factory fərqi nədir?” sualına cavab ver. **Biri dəyişən davranışın icrasını ayırır, o biri uyğun obyektin alınmasını təşkil edir.**

Yeni pattern istifadə edəndə onun çıxarılması ilə hansı konkret problemin geri qayıdacağını izah edə bil. Cavab yalnız “kod çirkin olar” olmamalıdır.

Əsas nəticə: pattern seçimi problemin yeri və dəyişən hissəsi ilə əsaslandırılır. Adı əzbərləmək həmin qərarı əvəz etmir.